

DEPLOYMENT

Development containers

Run Claude Code inside a dev container for consistent, isolated environments across your team.

A **development container**, or dev container, lets you define an identical, isolated environment that every engineer on your team can run. With Claude Code installed in that container, commands Claude runs execute inside it rather than on the host machine, while edits to your project files appear in your local repository as you work.

This page covers **installing Claude Code in a dev container**, then a set of self-contained configuration topics: persisting authentication across rebuilds, enforcing organization policy, restricting network egress, and running without permission prompts. Read the ones that match your setup.

⚠ While the dev container provides substantial protections, no system is completely immune to all attacks. When executed with `--dangerously-skip-permissions`, dev containers do not prevent a malicious project from exfiltrating anything accessible inside the container, including the Claude Code credentials stored in `~/.claude`. Only use dev containers when developing with trusted repositories, and monitor Claude's activities. Avoid mounting host secrets such as `~/.ssh` or cloud credential files into the container; prefer repository-scoped or short-lived tokens.

▶ How dev containers work with your editor

Add Claude Code to your dev container

Claude Code installs into any dev container through the **Claude Code Dev Container Feature**.

The settings work with any tool that supports the Dev Containers spec, such as VS Code, GitHub Codespaces, or JetBrains IDEs. The steps below use VS Code as an example.

When you open the container in VS Code or Codespaces, the feature also adds the Claude Code VS Code extension; other editors ignore that part.

💡 New to dev containers? The [VS Code Dev Containers tutorial](#) walks through installing Docker, the extension, and opening your first container. For a fuller hardened example with a firewall and persistent volumes, see [Try the reference container](#).

1 Create or update devcontainer.json

Save the following as `.devcontainer/devcontainer.json` in your repository, or add the `features` block to your existing file.

The version tag at the end, such as `:1.0`, pins the feature's install script, not the Claude Code release. The feature installs the latest Claude Code, and Claude Code auto-updates itself inside the container by default.

To pin the CLI version or disable auto-update, see [Enforce organization policy](#).

`.devcontainer/devcontainer.json`

```
{
  "image": "mcr.microsoft.com/devcontainers/base:ubuntu",
  "features": {
    "ghcr.io/anthropics/devcontainer-features/claude-code:1.0": {}
  }
}
```

Replace the `image` line with your project's base image or remove it if your existing file uses a Dockerfile.

2 Rebuild the container

Open the VS Code Command Palette with `Cmd+Shift+P` on Mac or `Ctrl+Shift+P` on Windows and Linux, and run **Dev Containers: Rebuild Container**.

For other tools, follow that tool's rebuild action: see [rebuilding in GitHub Codespaces](#), the [Dev Containers CLI](#), or your IDE's dev container documentation.

3 Sign in to Claude Code

Open a terminal in the rebuilt container and run `claude`, then follow the authentication prompt.

What you see at the authentication prompt depends on your provider:

- **Anthropic:** sign in through a browser with your Claude or Anthropic Console account
- **Amazon Bedrock, Google Vertex AI, or Microsoft Foundry:** Claude Code uses your cloud provider credentials, with no browser prompt

For cloud providers, pass credentials into the container as environment variables through `containerEnv`, a Codespaces secret, or your cloud's workload identity rather than mounting credential files from the host. See [Amazon Bedrock](#), [Google Vertex AI](#), or [Microsoft Foundry](#) for the credential chain Claude Code reads.

See [Choose your API provider](#) to decide which path fits your organization.

❗ If the browser sign-in completes but the callback never reaches the container, copy the code shown in the browser and paste it at the `Paste code here if prompted` prompt in the terminal. This can happen when the editor's port forwarding doesn't route the localhost callback.

Persist authentication and settings across rebuilds

By default, the container's home directory is discarded on rebuild, so engineers must sign in again each time. Claude Code stores its authentication token, user settings, and session history under `~/.claude`. Mount a named volume at that path to keep this state across rebuilds.

The following example mounts a volume at the home directory of the `node` user:

devcontainer.json

```
"mounts": [  
  "source=claude-code-config,target=/home/node/.claude,type=volume"  
]
```

Replace `/home/node` with the home directory of your container's `remoteUser`. If you mount the volume somewhere other than `~/.claude`, set `CLAUDE_CONFIG_DIR` to the mount path so Claude Code reads and writes there.

To isolate state per project rather than sharing one volume across all repositories, include the `${devcontainerId}` variable in the source name. The [reference configuration](#) uses `source=claude-code-config-${devcontainerId}` for this purpose.

In GitHub Codespaces, `~/.claude` persists across stopping and starting a codespace, but is still cleared when you rebuild the container, so the volume mount above applies there too. To carry authentication across codespaces, store `ANTHROPIC_API_KEY` or a `CLAUDE_CODE_OAUTH_TOKEN` from `claude setup-token` as a **Codespaces secret**; Codespaces makes secrets available as environment variables inside the container automatically.

Enforce organization policy

A dev container is a convenient place to apply organization policy, because the same image and configuration run on every engineer's machine.

Claude Code reads `/etc/claude-code/managed-settings.json` on Linux and applies it at the highest precedence in the **settings hierarchy**, so values there override anything an engineer sets in `~/.claude` or the project's `.claude/` directory. Copy the file into place from your Dockerfile:

Dockerfile

```
RUN mkdir -p /etc/claude-code
COPY managed-settings.json /etc/claude-code/managed-settings.json
```

Because the Dockerfile lives in the repository, anyone with write access can change or remove this step. For policy that engineers cannot bypass by editing repository files, deliver managed settings through **server-managed settings** or your MDM instead. See **managed settings files** for the available keys and the other delivery paths.

To set **environment variables** that apply to every Claude Code session in the container, add them to `containerEnv` in your `devcontainer.json`. The following example opts out of telemetry and error reporting and prevents Claude Code from auto-updating after install:

devcontainer.json

```
"containerEnv": {
  "CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC": "1",
  "DISABLE_AUTOUPDATER": "1"
}
```

The Dev Container Feature always installs the latest Claude Code release. To pin a specific Claude Code version for reproducible builds, install it from your Dockerfile with `npm install -g @anthropic-ai/claude-code@X.Y.Z` instead of using the feature, and set `DISABLE_AUTOUPDATER` as

shown above.

For the full list of policy controls including permission rules, tool restrictions, and MCP server allowlists, see [Set up Claude Code for your organization](#).

To make **MCP servers** available inside the container, define them at **project scope** in a `.mcp.json` file at the repository root so they are checked in alongside your dev container configuration. Install any binaries that local stdio servers depend on in your Dockerfile, and add remote server domains to your network allowlist.

Restrict network egress

You can limit the container's outbound traffic to only the domains Claude Code needs. See [Network access requirements](#) for the inference and authentication domains, and [Telemetry services](#) for the optional telemetry and error reporting connections and how to disable them.

The reference container includes an `init-firewall.sh` script that blocks all outbound traffic except the domains Claude Code and your development tools need. Running a firewall inside a container requires extra permissions, so the reference adds the `NET_ADMIN` and `NET_RAW` capabilities through `runArgs`. The firewall script and these capabilities are not required for Claude Code itself: you can leave them out and rely on your own network controls instead.

Run without permission prompts

Because the container runs Claude Code as a non-root user and confines command execution to the container, you can pass `--dangerously-skip-permissions` for unattended operation. The CLI rejects this flag when launched as root, so confirm `remoteUser` is set to a non-root account.

Skipping permission prompts removes your opportunity to review tool calls before they run. Claude can still modify any file in the bind-mounted workspace, which appears directly on your host, and reach anything the container's network policy allows. Pair this flag with the [network egress restrictions](#) above to limit what a bypassed session can reach.

If you want fewer prompts without disabling safety checks, consider **auto mode** instead, which has a classifier review actions before they run. To prevent engineers from using `--dangerously-skip-permissions` at all, set `permissions.disableBypassPermissionsMode` to `"disable"` in [managed settings](#).

Try the reference container

The `anthropics/claude-code` repository includes an example dev container that combines the CLI, the egress firewall, persistent volumes, and a Zsh-based shell. It is provided as a working example rather than a maintained base image; use it to see how the pieces fit together before applying them to your own configuration.

1 Install prerequisites

Install VS Code and the Dev Containers extension.

2 Clone the reference

Clone the Claude Code repository and open it in VS Code.

3 Reopen in container

When prompted, click **Reopen in Container**, or run **Dev Containers: Reopen in Container** from the Command Palette.

4 Start Claude Code

Once the container finishes building, open a terminal with `Ctrl+`` and run `claude` to sign in and start your first session.

To use this configuration with your own project, copy the `.devcontainer/` directory into your repository and adjust the Dockerfile for your toolchain, or return to Add Claude Code to your dev container to add only the feature to a setup you already have.

The reference configuration consists of three files. None of them are required when you add Claude Code to your own dev container through the feature, but they show one way to combine the pieces.

File	Purpose
<code>devcontainer.json</code>	Volume mounts, <code>runArgs</code> capabilities, VS Code extensions, and <code>containerEnv</code>
<code>Dockerfile</code>	Base image, development tools, and the Claude Code install
<code>init-firewall.sh</code>	Blocks all outbound network traffic except the allowed domains

Next steps

Once Claude Code is running in your dev container, the pages below cover the rest of an organization rollout: choosing an authentication path, delivering managed policy outside the repository, monitoring usage, and understanding what Claude Code stores and sends.

- **Set up Claude Code for your organization**: choose an authentication provider, decide how policy reaches devices, and plan the rollout
- **Server-managed settings**: deliver managed policy from the Claude.ai admin console so engineers cannot bypass it by editing repository files
- **Monitor usage and audit activity**: export OpenTelemetry metrics and review what your team is running
- **Network access requirements**: the full domain allowlist for proxies and firewalls
- **Telemetry services and opt-out**: what Claude Code sends by default and the environment variables that disable it
- **Explore the `.claude` directory**: what the volume mount holds, including credentials, settings, and session history
- **Sandbox environments**: compare dev containers with the built-in Bash sandbox, custom containers, and VMs
- **Security model**: how Claude Code's permission system, sandboxing, and prompt-injection protections fit together
- **Permission modes**: the full range from plan mode to auto mode to bypass, and when to use each